

Marmara University

Electrical and Electronics Engineering Department

Spring 2024

EE4051. Electromagnetic Waves

Simulaton Project Report

Group Members

Feyzullah Oğuz	150720035	
Mehmet Akif Türkmenoğlu	150720061	

The general form of the EM wave equation is given (for the electric field) by,

$$\vec{\nabla}^2\vec{E} = \mu\sigma\frac{\partial^2\vec{E}}{\partial t^2} \tag{1}$$

A similar equation can be written for the magnetic field. For a charge-free non-conducting space, the equation reduces to (in rectangular coordinates),

$$\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}\right)\vec{E} = \mu\varepsilon\frac{\partial^2\vec{E}}{\partial t^2} \tag{2}$$

In general, the electric field is described by a vector of three-components as,

$$\vec{E}(x,y,z,t) = E_x(x,y,z,t)\vec{a}_x + E_y(x,y,z,t)\vec{a}_y + E_z(x,y,z,t)\vec{a}_z \tag{3}$$

Here, a simplified case is considered, where the electric field has only on component (E_z) varying only in x and y dimensions. Equation 2 becomes scalar, and is written as,

$$\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}\right)E = \mu\sigma\frac{\partial^2\vec{E}}{\partial t^2} \tag{4}$$

In this project, the Finite-Difference Time-Domain (FDTD) method will be used. The FDTD method is one of the most popular numerical methods in the field of computational electromagnetics [1, 2]. Its main advantages are (1) easy to formulate, (2) suitable for many structures (3) flexible for modifications and (4) robust.

The FDTD-based simulation is performed in the following steps.

1) Discretize the given structure (make a grid) using a suitable step size in both dimensions (Δx and Δy). The accuracy of the results improves with smaller step sizes.

2) Approximate the partial derivatives in equation 4 by the following relations.

$$\frac{\partial^2}{\partial x^2} = \frac{E(i+1,j)-2E(i,j)+E(i-1,j)}{\Delta x^2} \quad (5)$$

$$\frac{\partial^2}{\partial y^2} = \frac{E(i,j+1)-2E(i,j)+E(i,j-1)}{\Delta y^2} \quad (6)$$

$$\frac{\partial^2 E}{\partial t^2} = \frac{E^{n+1}-2E^n+E^{n-1}}{\Delta t^2} \quad (7)$$

where i and j are the indices along the x and the y-axis, respectively. The index n is for time.
Substituting equations 5, 6 and 7 in equation 4, and using $\Delta x^2 = \Delta y^2 = \Delta^2$, we get

$$E^{n+1}(i,j) = 2\,E^n(i,j)-E^{n-1}(i,j)+\frac{\Delta t^2}{\mu\varepsilon\Delta^2}\,[E^n(i+1,j)+E^n(i-1,j)+E^n(i,j+1)+E^n(i,j-1)-4E^n(i,j)] \quad (8)$$

This is called the "update equation". It represents the time evolution of the wave at any given space point.

3) Implement the initial and boundary conditions and the material properties of the solution space.

4) Execute the time domain algorithm (equation 8) over all points in the specified space (excluding the boundaries) from $t = 0$ to any specified time duration (time marching).

Procedure

We first generate a matrix as 500x500 cm^2 and apply the equation (8) to represent the wave at any given space point. Then add the boundary conditions to each edge of the square.

We define a source that called Gaussian Pulse and to generate signal we use the formula as follows:

$$E_s(t) = e^{[\frac{t-t_0}{\delta}]^2} \quad (9)$$

where t_0 is a time delay and δ is a parameter that controls the width of the pulse.

We are asked to observe the propagation of the electromagnetic wave and make studies and analysis.

We determine time as $25*t_0$. Larger than that time values results the wave hits to the boundaries. When it happened, wave reflects and comes back but this reflection is complex to implement and we don't need them for this project. It is enough to observe the wave up to the boundaries.

In matlab, we used the same code for each step size. So we want to explain the code with common important points. We should point out that also there are minor differences between codes like determining the location of gaussian pulse (source) as points or determining the location of observation points to calculate the speed. In the code you can see these differences.

→First of all, we define the simulation parameters like dimension of the square matrix, speed of light, time step (t_0), delta (δ), total simulation time, constants (). Then generate the grid and apply boundary conditions. We define the gaussian pulse and implement it for the source point. We used the "update equation" to define electric field for each matrix point using for loop.

In order to get the video to show propagation of the wave in 2D, we create an object and use 'VideoWriter' code to save video to computer. We convert this video to GIF for convenience and add them to the report in each part.

We specify two observation points and use them in order to plot the electric field and gaussian pulse graphs. The graphs will be shown after the code in every part.

We don't need to put captions under every figure because each figure has a title says what it is.

1. Wave Propagation in Different Step Sizes with epsilon_r =1

1.1. Step Size 5 Cm with epsilon_r = 1

```
clear

%define parameters
L = 500;
dx = 5;
dy = 5;
c = 3e8;
dt = (0.5 * dx) / c; % Δt ≤ 0.5Δ/c
t0 = 0.05e-6; % time delay
delta = t0 / 3;
T = 15 * t0; % total time
mu_r = 1;
mu = mu_r * 4 * pi * 1e-7;
epsilon_r = 1;
epsilon = epsilon_r * 8.854*1e-12;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

%create and save video
video = VideoWriter('stepsize5.mp4', 'MPEG-
4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;
E_observationpoint = zeros(1,
ceil(T/dt)+1);
E_observationpoint1 = zeros(1,
ceil(T/dt)+1);
E_source = zeros(1, ceil(T/dt)+1);

% pulse equations
for t = 0:dt:T
    E_next = zeros(size(E));

    E_pulse = exp(-((t - t0) / delta)^2);
% at t=0
    E(50,50) = E_pulse;
    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + (dt^2) / (mu * epsilon *
dx^2) * (E(i+1,j) ...
                + E(i-1,j) + E(i,j+1) +
E(i,j-1) - 4 * E(i,j));
        end
    end
    % observation points
    E_observationpoint(ceil(t/dt)+1) =
E(50,60);
    E_observationpoint1(ceil(t/dt)+1) =
E(50,70);
    E_source(ceil(t/dt)+1) = E(50,50);
    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;

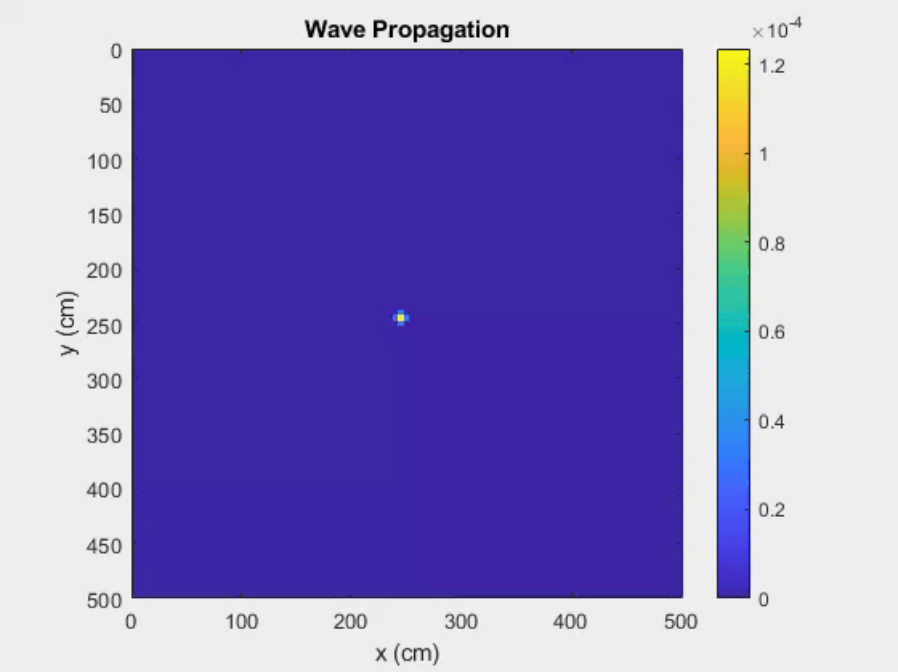
    writeVideo(video, getframe(gcf));
end
```

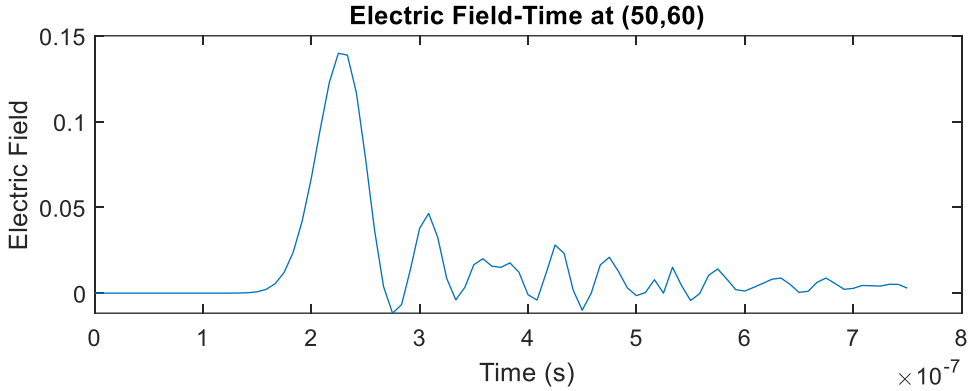
```
close(video);

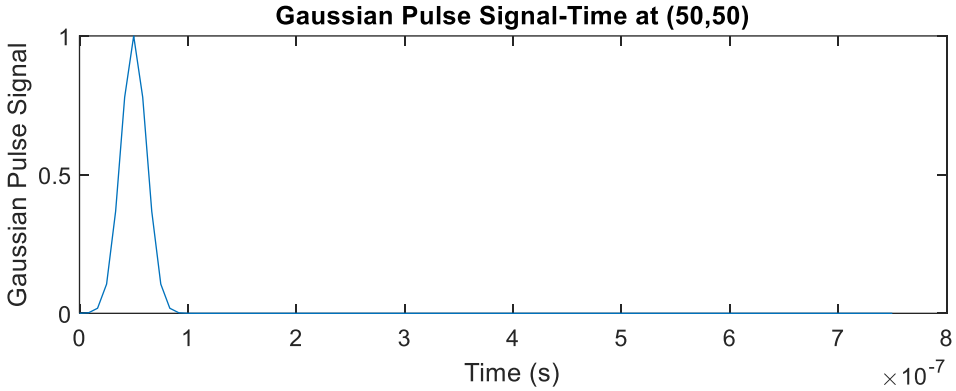
subplot(2,1,1)
plot(0:dt:T, E_observationpoint);
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (50,60)');

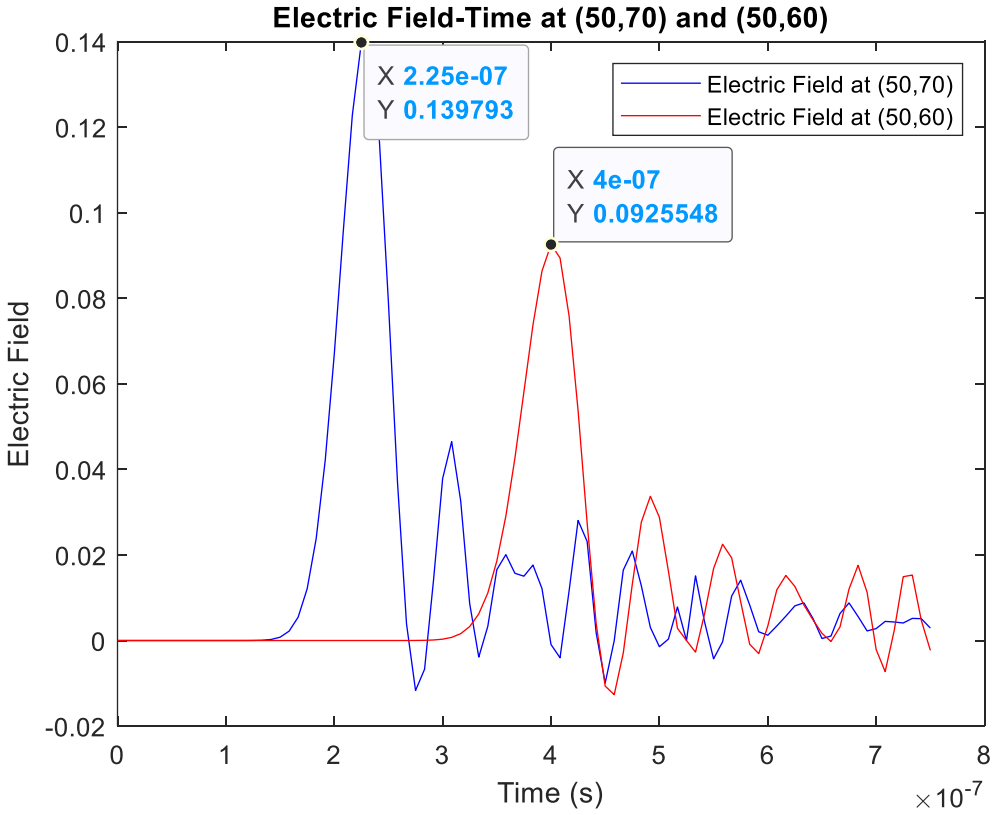

subplot(2,1,2)
plot(0:dt:T, E_source);
xlabel('Time (s)');
ylabel('Gaussian Pulse Signal');
title('Gaussian Pulse Signal-Time at (50,50)');

figure()
subplot()
plot(0:dt:T,
E_observationpoint,'b',0:dt:T,E_observationpoint1,'r');
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (50,70) and (50,60)');
legend('Electric Field at (50,70)','Electric Field at (50,60)')
```




Electric Field-Time at (50,60)
Electric Field
Time (s)
×10^-7


Gaussian Pulse Signal-Time at (50,50)
Gaussian Pulse Signal
Time (s)
×10^-7


Electric Field-Time at (50,70) and (50,60)
X 2.25e-07
Y 0.139793
X 4e-07
Y 0.0925548
Electric Field at (50,70)
Electric Field at (50,60)
Electric Field
Time (s)
×10^-7

1.2. Step Size 2 Cm with epsilon_r = 1

```
clear
%define parameters
L = 500;
dx = 2;
dy = 2;
c = 3e8;
dt = (0.5 * dx) / c; % Δt ≤ 0.5Δ/c
t0 = 0.05e-6; % time delay
delta = t0 / 3;
T = 15 * t0; % total time
mu_r = 1;
mu = mu_r * 4 * pi * 1e-7;

epsilon_r = 1;
epsilon = epsilon_r * 8.854*1e-12;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

%create and save video
video = VideoWriter('stepsize2.mp4', 'MPEG-
4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;
E_observationpoint = zeros(1,
ceil(T/dt)+1);
E_observationpoint1 = zeros(1,
ceil(T/dt)+1);
E_source = zeros(1, ceil(T/dt)+1);

% pulse equations
for t = 0:dt:T
    E_next = zeros(size(E));

    E_pulse = exp(-((t - t0) / delta)^2);
% at t=0
    E(125,125) = E_pulse;
    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + (dt^2) / (mu * epsilon *
dx^2) * (E(i+1,j) ...
                + E(i-1,j) + E(i,j+1) +
E(i,j-1) - 4 * E(i,j));
        end
    end

    % observation points
    E_observationpoint(ceil(t/dt)+1) =
E(125,150);
    E_observationpoint1(ceil(t/dt)+1) =
E(125,175);
    E_source(ceil(t/dt)+1) = E(125,125);
    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;

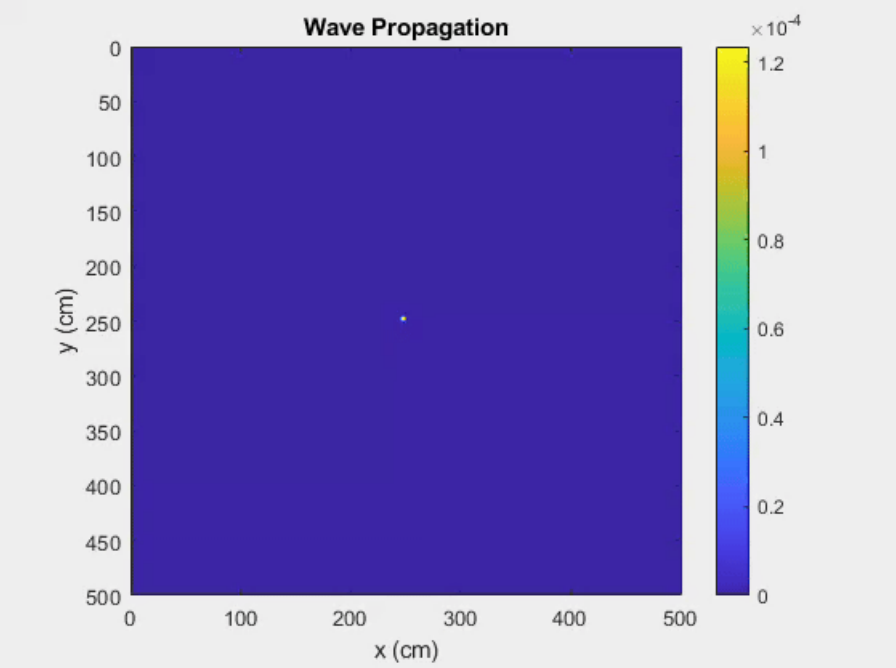
    writeVideo(video, getframe(gcf));
end

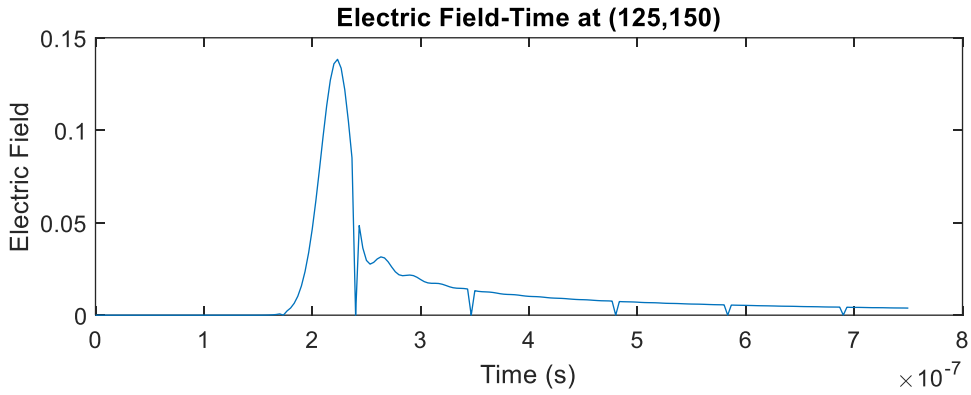
close(video);
```

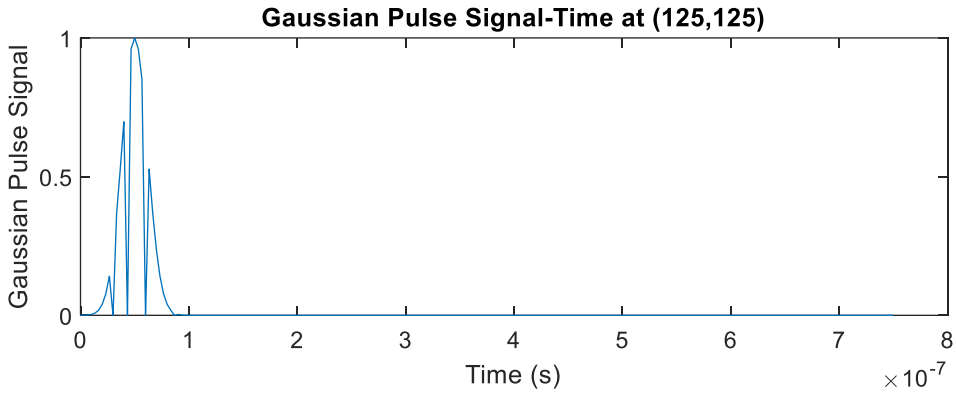
```
subplot(2,1,1)
plot(0:dt:T, E_observationpoint);
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (125,150)');

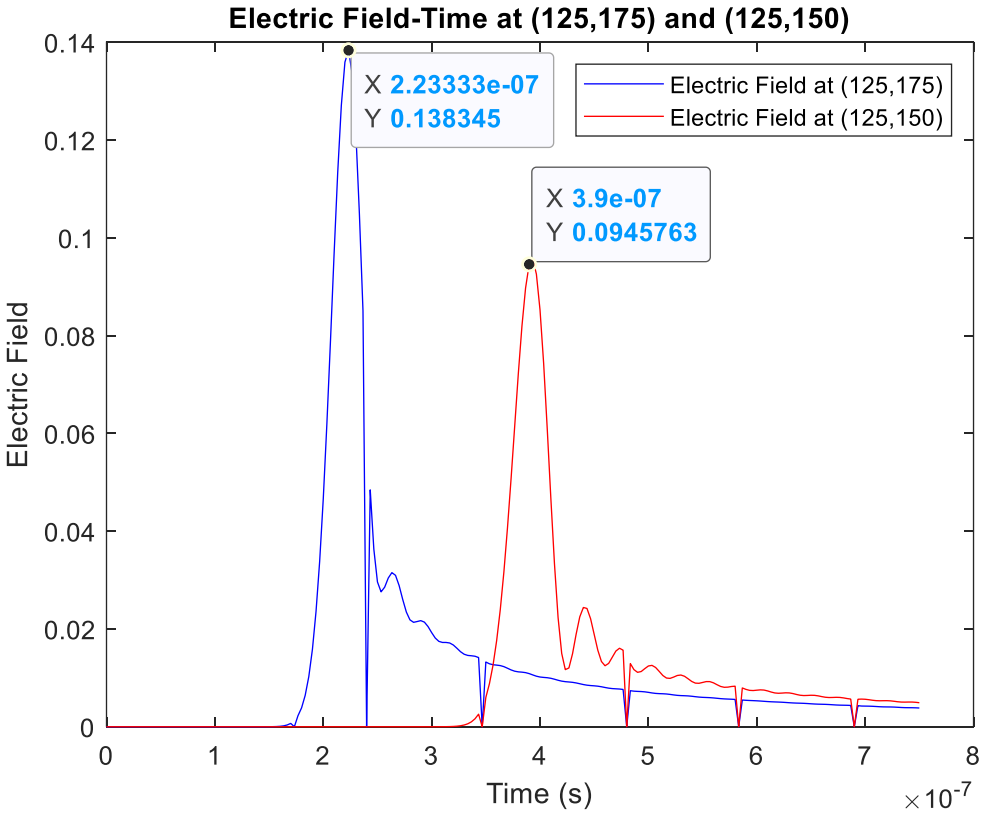

subplot(2,1,2)
plot(0:dt:T, E_source);
xlabel('Time (s)');
ylabel('Gaussian Pulse Signal');
title('Gaussian Pulse Signal-Time at (125,125)');


figure()
subplot()
plot(0:dt:T,
E_observationpoint,'b',0:dt:T,E_observationpoint1,'r');
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (125,175) and (125,150)');
legend('Electric Field at (125,175)','Electric
Field at (125,150)')
```




Electric Field-Time at (125,150)
Electric Field
Time (s)
×10^-7


Gaussian Pulse Signal-Time at (125,125)
Gaussian Pulse Signal
Time (s)
×10^-7


Electric Field-Time at (125,175) and (125,150)
X 2.23333e-07
Y 0.138345
X 3.9e-07
Y 0.0945763
Electric Field at (125,175)
Electric Field at (125,150)
Electric Field
Time (s)
×10^-7

1.3. Step Size 1 Cm with epsilon_r = 1

```
clear
%define parameters
L = 500;
dx = 1;
dy = 1;
c = 3e8;
dt = (0.5 * dx) / c; % Δt ≤ 0.5Δ/c
t0 = 0.05e-6; % time delay
delta = t0 / 3;
T = 15 * t0; % total time
mu_r = 1;
mu = mu_r * 4 * pi * 1e-7;

epsilon_r = 1;
epsilon = epsilon_r * 8.854*1e-12;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

%create and save video
video = VideoWriter('stepsize1.mp4', 'MPEG-
4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;
E_observationpoint = zeros(1,
ceil(T/dt)+1);
E_observationpoint1 = zeros(1,
ceil(T/dt)+1);
E_source = zeros(1, ceil(T/dt)+1);

% pulse equations
for t = 0:dt:T
    E_next = zeros(size(E));

    E_pulse = exp(-((t - t0) / delta)^2);
% at t=0
    E(250,250) = E_pulse;
    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + (dt^2) / (mu * epsilon *
dx^2) * (E(i+1,j) ...
                + E(i-1,j) + E(i,j+1) +
E(i,j-1) - 4 * E(i,j));
        end
    end

    % observation points
    E_observationpoint(ceil(t/dt)+1) =
E(250,300);
    E_observationpoint1(ceil(t/dt)+1) =
E(250,350);
    E_source(ceil(t/dt)+1) = E(250,250);
    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;

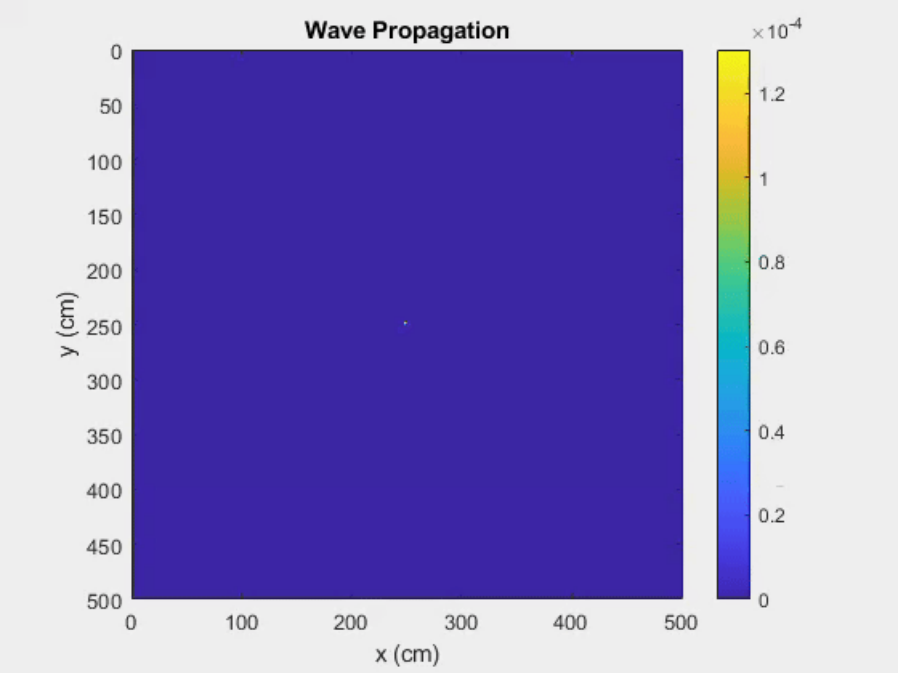
    writeVideo(video, getframe(gcf));
end

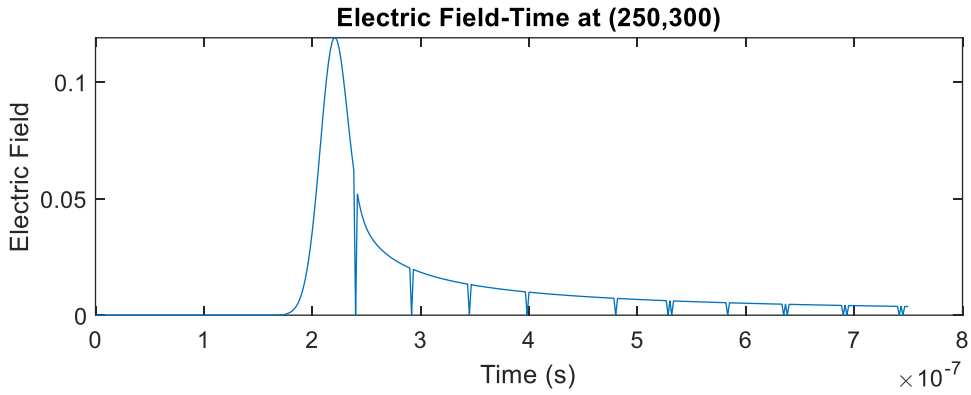
close(video);
```

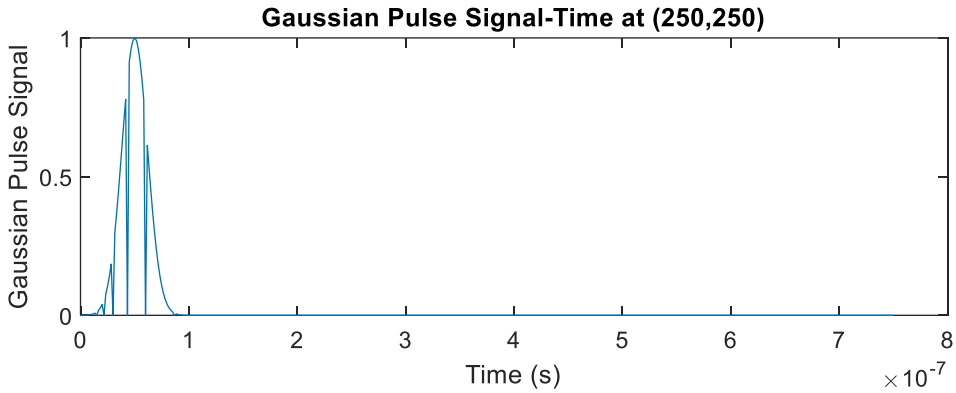
```
subplot(2,1,1)
plot(0:dt:T, E_observationpoint);
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (250,300)');

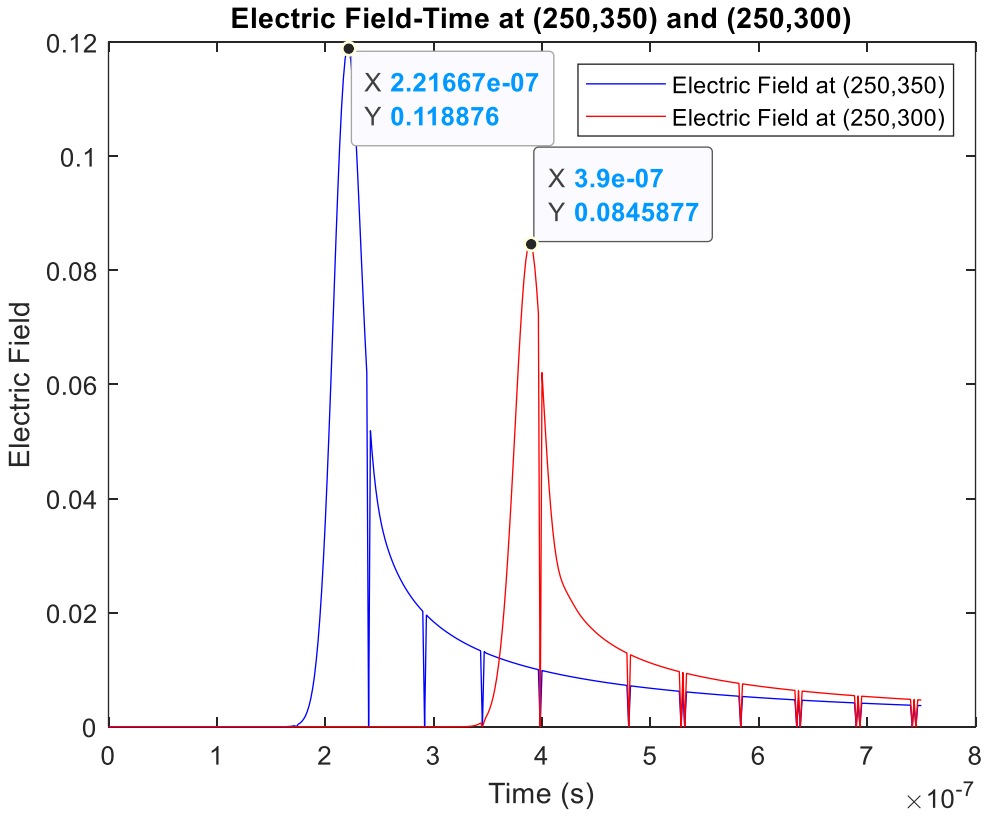

subplot(2,1,2)
plot(0:dt:T, E_source);
xlabel('Time (s)');
ylabel('Gaussian Pulse Signal');
title('Gaussian Pulse Signal-Time at (250,250)');


figure()
subplot()
plot(0:dt:T,
E_observationpoint,'b',0:dt:T,E_observationpoint1,'r');
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (250,350) and (250,300)');
legend('Electric Field at (250,350)','Electric
Field at (250,300)')
```




Electric Field-Time at (250,300)
Electric Field
0.1
0.05
0
0
1
2
3
4
5
6
7
8
Time (s)
×10^-7


Gaussian Pulse Signal-Time at (250,250)
Gaussian Pulse Signal
1
0.5
0
0
1
2
3
4
5
6
7
8
Time (s)
×10^-7


Electric Field-Time at (250,350) and (250,300)
X 2.21667e-07
Y 0.118876
X 3.9e-07
Y 0.0845877
Electric Field at (250,350)
Electric Field at (250,300)
Electric Field
0.12
0.1
0.08
0.06
0.04
0.02
0
0
1
2
3
4
5
6
7
8
Time (s)
×10^-7

1.4. Step Size 0.5 Cm with epsilon_r = 1

```
clear
%define parameters
L = 500;
dx = 0.5;
dy = 0.5;
c = 3e8;
dt = (0.5 * dx) / c; % Δt ≤ 0.5Δ/c
t0 = 0.05e-6; % time delay
delta = t0 / 3;
T = 15 * t0; % total time
mu_r = 1;
mu = mu_r * 4 * pi * 1e-7;

epsilon_r = 1;
epsilon = epsilon_r * 8.854*1e-12;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

%create and save video
video = VideoWriter('stepsize05.mp4',
'MPEG-4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;
E_observationpoint = zeros(1,
ceil(T/dt)+1);
E_observationpoint1 = zeros(1,
ceil(T/dt)+1);
E_source = zeros(1, ceil(T/dt)+1);

% pulse equations
for t = 0:dt:T
    E_next = zeros(size(E));

    E_pulse = exp(-((t - t0) / delta)^2);
% at t=0
    E(500,500) = E_pulse;
    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + (dt^2) / (mu * epsilon *
dx^2) * (E(i+1,j) ...
                + E(i-1,j) + E(i,j+1) +
E(i,j-1) - 4 * E(i,j));
        end
    end

    % observation points
    E_observationpoint(ceil(t/dt)+1) =
E(500,600);
    E_observationpoint1(ceil(t/dt)+1) =
E(500,700);
    E_source(ceil(t/dt)+1) = E(500,500);
    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;

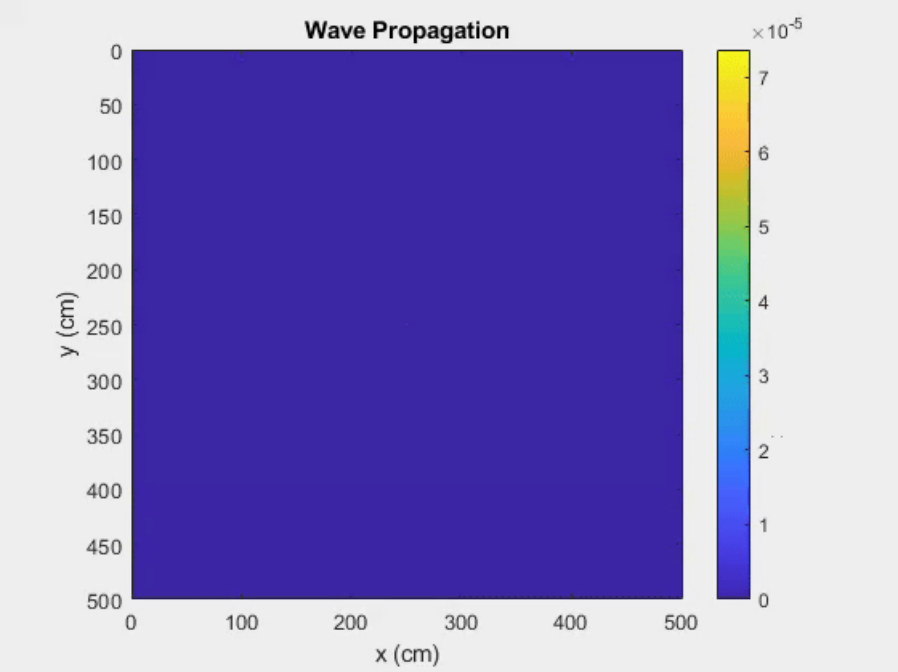
    writeVideo(video, getframe(gcf));
end

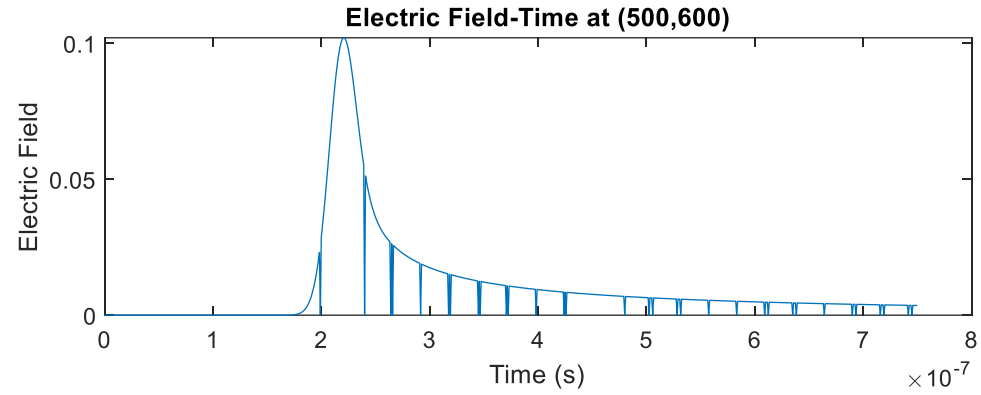
close(video);
```

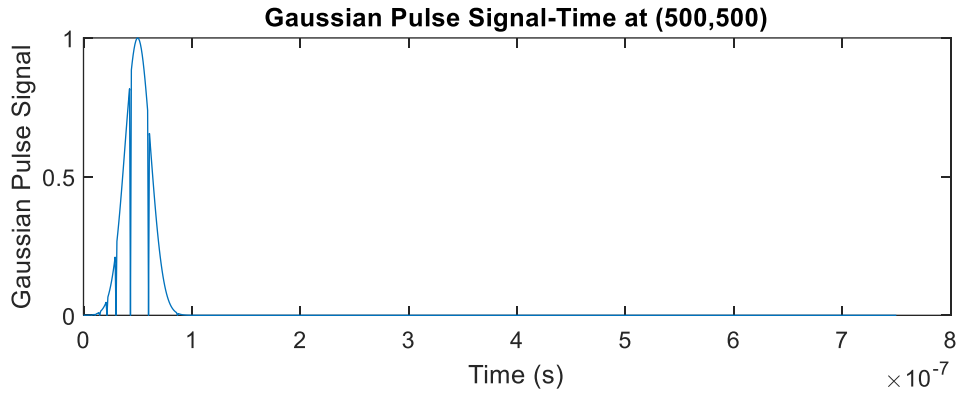
```
subplot(2,1,1)
plot(0:dt:T, E_observationpoint);
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (500,600)');

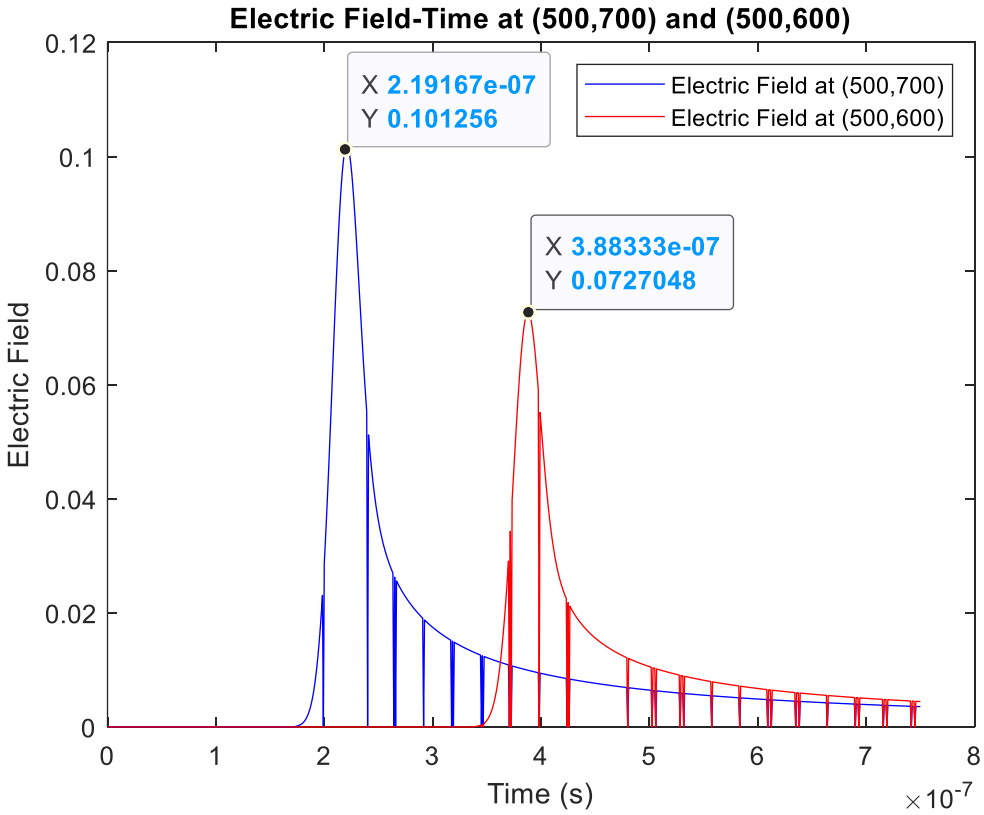

subplot(2,1,2)
plot(0:dt:T, E_source);
xlabel('Time (s)');
ylabel('Gaussian Pulse Signal');
title('Gaussian Pulse Signal-Time at (500,500)');


figure()
subplot()
plot(0:dt:T,
E_observationpoint,'b',0:dt:T,E_observationpoint1,'r');
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (500,700) and (500,600)');
legend('Electric Field at (500,700)','Electric
Field at (500,600)')
```




Electric Field-Time at (500,600)
Electric Field
Time (s)
×10^-7


Gaussian Pulse Signal-Time at (500,500)
Gaussian Pulse Signal
Time (s)
×10^-7


Electric Field-Time at (500,700) and (500,600)
X 2.19167e-07
Y 0.101256
X 3.88333e-07
Y 0.0727048
Electric Field at (500,700)
Electric Field at (500,600)
Electric Field
Time (s)
×10^-7

2. Wave Propagation in Step Size 5 Cm with epsilon_r = 2.5

```
clear
%define parameters
L = 500;
dx = 5;
dy = 5;
c = 3e8;
dt = (0.5 * dx) / c; % Δt ≤ 0.5Δ/c
t0 = 0.05e-6; % time delay
delta = t0 / 3;
T = 25 * t0; % total time
mu_r = 1;
mu = mu_r * 4 * pi * 1e-7;
epsilon_r = 2.5;
epsilon = epsilon_r * 8.854*1e-12;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

%create and save video
video = VideoWriter('stepsize5.mp4', 'MPEG-
4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;
E_observationpoint = zeros(1,
ceil(T/dt)+1);
E_observationpoint1 = zeros(1,
ceil(T/dt)+1);
E_source = zeros(1, ceil(T/dt)+1);

% pulse equations
for t = 0:dt:T
    E_next = zeros(size(E));

    E_pulse = exp(-((t - t0) / delta)^2);
% at t=0
    E(50,50) = E_pulse;
    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + (dt^2) / (mu * epsilon *
dx^2) * (E(i+1,j) ...
                + E(i-1,j) + E(i,j+1) +
E(i,j-1) - 4 * E(i,j));
        end
    end

    % observation points
    E_observationpoint(ceil(t/dt)+1) =
E(50,60);
    E_observationpoint1(ceil(t/dt)+1) =
E(50,70);
    E_source(ceil(t/dt)+1) = E(50,50);
    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;

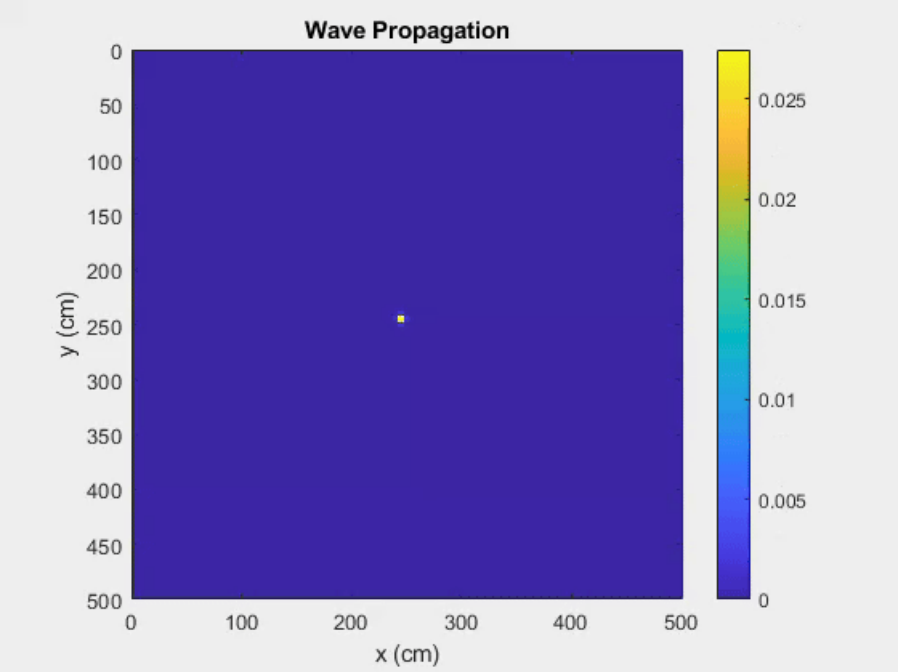
    writeVideo(video, getframe(gcf));
end

close(video);
```

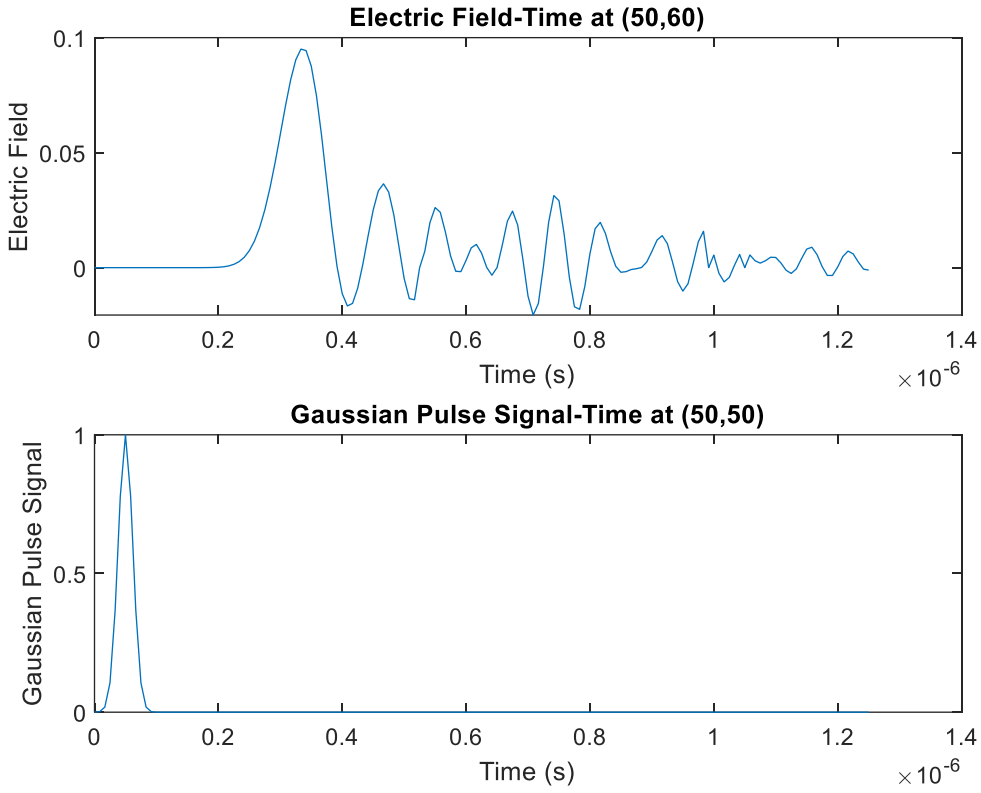
```
subplot(2,1,1)
plot(0:dt:T, E_observationpoint);
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (50,60)');

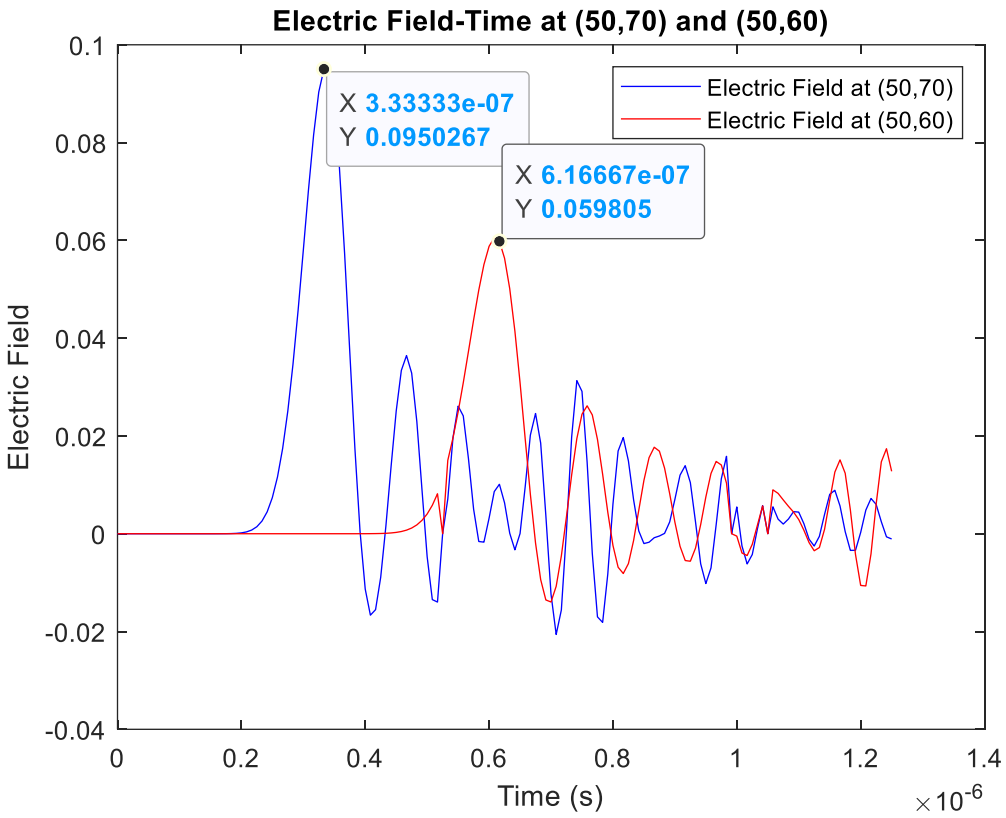

subplot(2,1,2)
plot(0:dt:T, E_source);
xlabel('Time (s)');
ylabel('Gaussian Pulse Signal');
title('Gaussian Pulse Signal-Time at (50,50)');

figure()
plot(0:dt:T,
E_observationpoint,'b',0:dt:T,E_observationpoint1,'r');
xlabel('Time (s)');
ylabel('Electric Field');
title('Electric Field-Time at (50,70) and (50,60)');
legend('Electric Field at (50,70)','Electric
Field at (50,60)')
```



We simulate the wave propagation with epsilon_r =2.5. Wave struggles to propagate in epsilon_r =2.5 more than propagation in epsilon_r = 1. We give the wave propagation simulation just in step size 5 cm not the other step sizes.





3. Wave Propagation in Fig 2

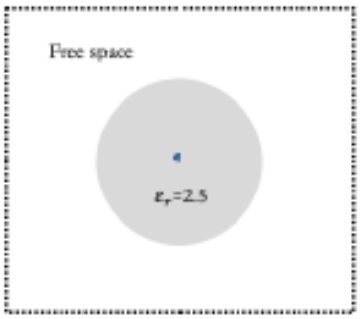


Fig 2.Structure under study

```
% simulation parameters
L = 500;
dx = 5;
dy = 5;
dt = 0.5 * min(dx, dy) / (3e8 / sqrt(2.5));
% Time step (s), adjusted for εr=2.5
t0 = 0.05e-6; % time delay
delta = t0 / 3;
T = 20 * t0; % total time
c = 3e8;
epsilon_r_free_space = 1;
epsilon_r_circle = 2.5;
mu_r = 1;
mu = mu_r*4*pi*1e-7;
epsilon_free_space = epsilon_r_free_space *
8.854*1e-12;
epsilon_circle = epsilon_r_circle *
8.854*1e-12;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);

% circle parameters
circle_center = [L/2, L/2];
circle_radius = L/4;

% check if point is inside the circle
inside_circle = @(i, j) (X(i, j) -
circle_center(1))^2 + (Y(i, j) -
circle_center(2))^2 <= circle_radius^2;

%initial conditions
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

% apply gaussian pulse
E_pulse = exp(-((0 - t0) / delta)^2); % at
t=0
E(round(L/(2*dx)), round(L/(2*dy))) =
E_pulse; %center


video = VideoWriter('soru9.mp4', 'MPEG-4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation with εr Circle');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;
for t = 0:dt:T
    E_next = zeros(size(E));
    E_pulse = exp(-((t - t0) / delta)^2);%
gaussian pulse at t=0
    E(round(L/(2*dx)), round(L/(2*dy))) =
E_pulse; % at center point


    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1

            % determine if the point is
inside the circle
            if inside_circle(i, j)
                epsilon = epsilon_circle;
            else
                epsilon =
epsilon_free_space;
            end

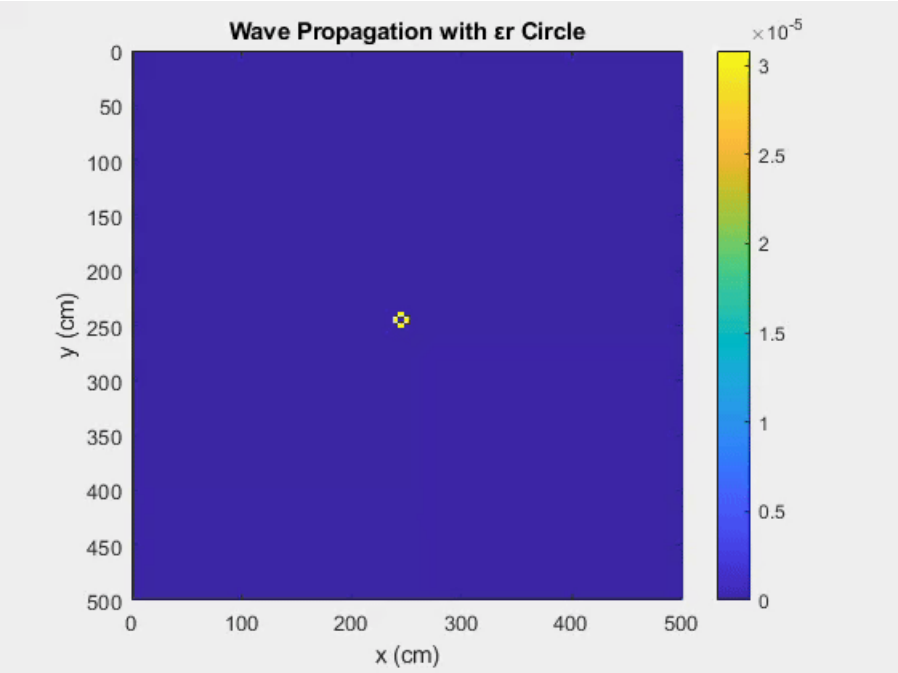
            % calculate the update
equation using the appropriate epsilon
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + ...
                (dt^2) / (mu * epsilon *
dx^2) * ...
                (E(i+1,j) + E(i-1,j) +
E(i,j+1) + E(i,j-1) - 4 * E(i,j));
        end
    end

    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;

    writeVideo(video, getframe(gcf));
end
close(video);
```


Wave Propagation with εr Circle
×10^-5
3
2.5
2
1.5
1
0.5
0
0
50
100
150
200
250
300
350
400
450
500
y (cm)
0
100
200
300
400
500
x (cm)

4. Propagation Based on Cosine - Gaussian Pulse

```
% simulation parameters
L = 500;
dx = 5;
dy = 5;
dt = 0.5 * min(dx, dy) / 3e8; % Time step
(s), using Δt ≤ 0.5Δ/c
t0 = 0.05e-6;
delta = t0 / 3;
T = 17 * t0; % total time
c = 3e8;
f_c = 2.5e9; % center frequency of the
cosine-Gaussian pulse
epsilon_r = 1;
mu_r = 1;
epsilon = epsilon_r * 8.854e-12;
mu = mu_r * 4 * pi * 1e-7;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);

% initial conditions
E = zeros(size(X));

% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

video = VideoWriter('soru10.mp4', 'MPEG-
4');
open(video);

% figures for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation');
colorbar;
axis equal;
axis([0 L 0 L]);
```

```
E_prev = E;

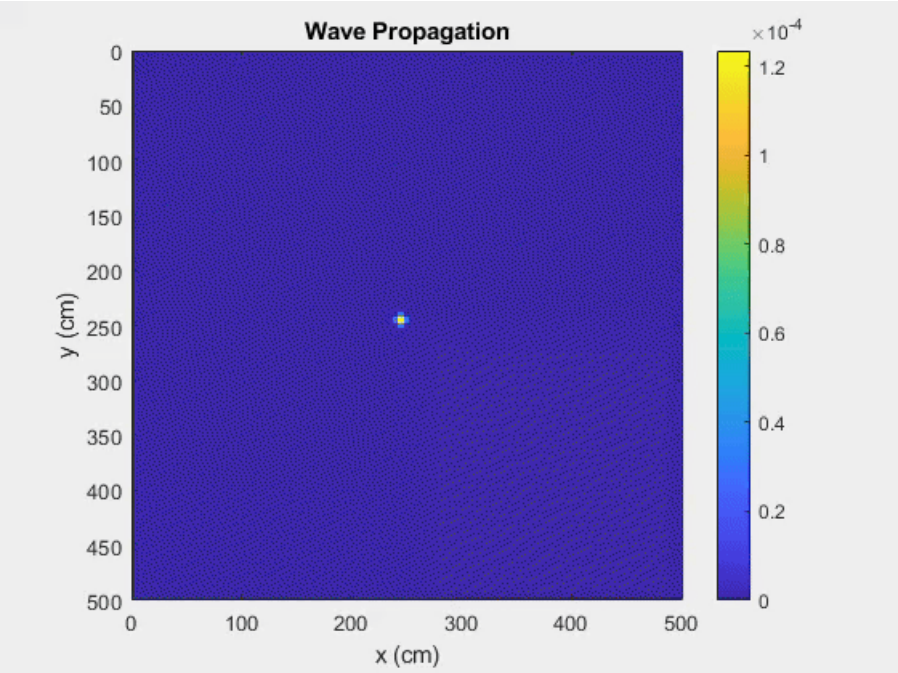
for t = 0:dt:T
    E_next = zeros(size(E));
    % cosine-Gaussian Pulse
    E_pulse = exp(-((t - t0) / delta)^2) *
cos(2 * pi * f_c * (t - t0));
    E(50,50) = E_pulse; %at center point

    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + ...
                (dt^2) / (mu * epsilon *
dx^2) * ...
                (E(i+1,j) + E(i-1,j) +
E(i,j+1) + E(i,j-1) - 4 * E(i,j));
        end
    end

    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;
    writeVideo(video, getframe(gcf));
end
close(video);
```



Understanding the differences between a Gaussian pulse and a cosine-Gaussian pulse is crucial in applications like electromagnetic wave propagation simulations. Each type of pulse exhibits distinct time-domain and frequency-domain characteristics, which can significantly impact the results of simulations. Here are the key differences along with observations and comments on their implications:

Gaussian Pulse

Temporal Profile: The Gaussian pulse features a smooth peak with maximum amplitude at the center, decreasing rapidly towards the edges. This creates a localized burst of energy in time.

Frequency Spectrum: It encompasses a wide range of frequencies, inversely related to the pulse width. Shorter duration Gaussian pulses have a broader bandwidth.

Propagation Characteristics: The broad bandwidth may lead to dispersion of different frequency components in various media, affecting the pulse shape and speed as it travels.

Cosine-Gaussian Pulse

Temporal Profile: The cosine-Gaussian pulse incorporates a cosine wave modulated within a Gaussian envelope, introducing periodic fluctuations in the pulse's amplitude over time.

Frequency Spectrum: It is concentrated around a specific center frequency, shaped by the Gaussian envelope, resulting in a narrower bandwidth compared to a pure Gaussian pulse.

Propagation Characteristics: The concentration of energy around the center frequency can lead to distinct reflection and interference patterns depending on the medium and boundary conditions.

5. Modifications

```
% simulation parameters
L = 500;
dx = 5;
dy = 5;
dt = 0.5 * min(dx, dy) / 3e8; % Time step
(s), using Δt ≤ 0.5Δ/c
t0 = 0.05e-6;
delta = t0 / 3;
T = 35 * t0; % total time
c = 3e8;
mu_r = 4;
mu = mu_r * 4 * pi * 1e-7;
epsilon_0 = 8.854*1e-12;
epsilon_r = ones(size(X));
half_grid = size(X,2)/2;
epsilon_r(:,1:half_grid) = 9;
epsilon = epsilon_r * epsilon_0;
% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);
E = zeros(size(X));
% boundary conditions
E(:, 1) = 0; E(:, end) = 0; E(1, :) = 0;
E(end, :) = 0;

E_prev = E;

video = VideoWriter('soru11.mp4', 'MPEG-
4');
open(video);
% Create figure for visualization
figure;
h = imagesc(x, y, E);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation with Spatially
Varying Epsilon_r');
```

```
colorbar;
axis equal;
axis([0 L 0 L]);

for t = 0:dt:T
    E_next = zeros(size(E));
    % gaussian pulse calculation
    E_pulse = exp(-((t - t0) / delta)^2);
    E(50,50) = E_pulse; % Gaussian pulse at
point (50, 50)
    for i = 2:size(E, 1)-1
        for j = 2:size(E, 2)-1
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + ...
                (dt^2) / (mu * epsilon(i,j)
* dx^2) * ...
                (E(i+1,j) + E(i-1,j) +
E(i,j+1) + E(i,j-1) - 4 * E(i,j));
        end
    end

    E_prev = E;
    E = E_next;

    % update boundary conditions
    E(:, 1) = 0; E(:, end) = 0; E(1, :) =
0; E(end, :) = 0;

    % update visualization
    set(h, 'CData', E);
    drawnow;
    writeVideo(video, getframe(gcf));
end
close(video);
```

5.1. Observations and Comments on Modifications

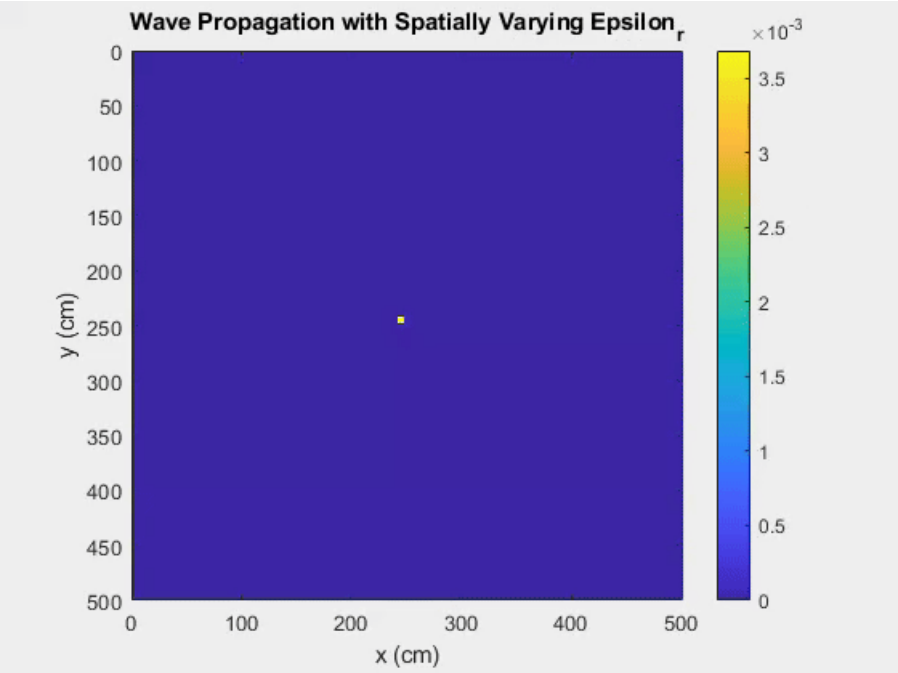
Energy Distribution: The cosine-Gaussian pulse can propagate energy more efficiently at its center frequency, leading to different reflection and absorption characteristics in the simulation environment. This focused energy distribution can be particularly useful for studying wave interactions with materials that have frequency-dependent properties.

Visualization of Waveforms: While a Gaussian pulse creates a smoothly advancing wavefront, the cosine-Gaussian pulse produces periodic modulations in the wavefront. This effect becomes apparent as the wave propagates and interacts with boundaries, potentially offering insights into the behavior of modulated signals in various media.

Frequency-Dependent Behaviors: The narrower frequency band of the cosine-Gaussian pulse allows for the examination of dispersion and reflection behaviors specific to that frequency range. This can be useful for analyzing how narrowband signals behave in communication systems and other applications.

Impact on Simulation Outcomes: Using a cosine-Gaussian pulse can highlight frequency-dependent features in your simulation results more prominently. For instance, the reflection and absorption rates at certain frequencies might differ significantly from those observed with a Gaussian pulse, providing valuable insights into the medium's response to different spectral components.

In your report, detailing these observations and the specific outcomes derived from your simulation results can elucidate the impact of pulse characteristics on electromagnetic wave behavior. It illustrates how the choice of pulse can play a vital role in understanding wave interactions within different environments.



Left side of the monitor displays the wave propagation in epsilon_r = 9 and right side displays the wave propagation in epsilon_r = 1 like previous simulations. We can see that epsilon value increases the wave is struggling more to propagate in the medium.

Increasing εr and μr values: To observe the impact of material properties on electromagnetic wave propagation, you can increase the values of εr and μr in simulation.By raising these values above 1, to for example 4 , you modify the medium's ability to store electric and magnetic energy, respectively. These changes will significantly affect wave velocity, wavelength, and reflection characteristics.

Wave Speed and Wavelength: As εr and μr alues are increased, you should observe a reduction in wave speed and a shortening of the wavelength. This indicates that the electromagnetic wave propagates slower through a medium with higher εr and μr values. Reflection and Refraction: Changes in boundary conditions and transitions between materials with different properties may alter reflection and refraction behaviors.Higher εr and μr values can increase the reflection rate.

Energy Distribution: The way the wave's energy is distributed and absorbed may change. Increased dielectric and magnetic properties can lead to different absorption characteristics of the wave within the medium.

6. Bonus Part

```
% simulation parameters
L = 500;
dx = 5;
dy = 5;
dt = 0.5 * min(dx, dy) / (3e8); % Time step
(s), using Δt ≤ 0.5Δ/c
t0 = 0.05e-6;
delta = t0 / 3;
T = 50 * t0; % total time
c = 3e8;
epsilon_r = 1;
mu_r = 1;
sigma = 0.01;
epsilon = epsilon_r * 8.854e-12;
mu = mu_r * 4 * pi * 1e-7;

% generate grid
x = 0:dx:L;
y = 0:dy:L;
[X, Y] = meshgrid(x, y);

%initial conditions
E = zeros(size(X)); % Initialize electric
field
E_prev = E; % Initialize previous electric
field
E_next = zeros(size(E)); % Placeholder for
the next electric field values

video = VideoWriter('soru12.mp4', 'MPEG-
4');
open(video);


% figures for visualization
figure;
h = imagesc(x, y, E);
colorbar;
axis equal;
axis([0 L 0 L]);
xlabel('x (cm)');
ylabel('y (cm)');
title('Wave Propagation with Medium Loss');
```

```
for t = dt:dt:T

    E_pulse = exp(-((t - t0) / delta)^2);

E(round(size(X,1)/2),round(size(Y,2)/2)) =
E_pulse;

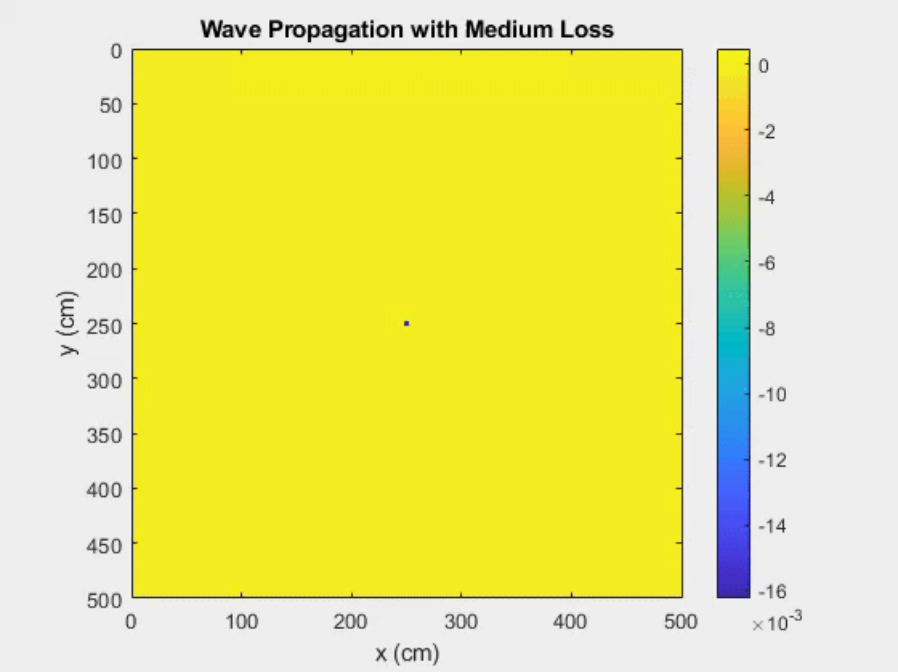
    % electric field with consideration
for medium loss
    for i = 2:(size(X,1)-1)
        for j = 2:(size(Y,2)-1)
            E_next(i,j) = 2 * E(i,j) -
E_prev(i,j) + ...
                (dt^2) / (mu * epsilon *
dx^2) * ...
                (E(i+1,j) + E(i-1,j) +
E(i,j+1) + E(i,j-1) - 4 * E(i,j)) - ...
                dt * (sigma / epsilon) *
E(i,j); %medium loss
        end
    end

    % update boundary conditions
    E_next(:,1) = 0; E_next(:,end) = 0;
E_next(1,:) = 0; E_next(end,:) = 0;

    E_prev = E;
    E = E_next;

    % visualization
    set(h, 'CData', E);
    drawnow;
    writeVideo(video, getframe(gcf));
end
close(video);
```

6.1. Observations and Comments on Bonus Part



This code initializes a simulation space, applies a Gaussian pulse as a source at the center, and updates the electric field over time while considering medium loss. The conductivity term σ causes the wave to attenuate as it propagates through the medium. You can change the sigma value to see how different levels of conductivity affect the wave's attenuation.